Lecture 7: Processes

Professor G. Sandoval

07

NYU

Some slides adapted by G. Sandoval for CS3224, from Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved. Also some Slides by Brendan Dolan-Gavitt and Stallings: Operatings Systems. Eight Edition.



Today

- **Processes**
- Process States
- Unix Processes
- XV6 Processes

How can 4 programs appear to run at once? One cpu executes one instruction at a time

The CPU combines their instructions

0%

The compiler creates extra CPUs

0%

Each program uses a different clock

0%

The CPU switches among the programs

0%

None of the above

0%

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

Virtualizing the CPU

```
1       #include <stdio.h>
2       #include <stdlib.h>
3       #include <sys/time.h>
4       #include <assert.h>
5       #include "common.h"
6
7       int
8       main(int argc, char *argv[])
9       {
10              if (argc != 2) {
11                      fprintf(stderr, "usage: cpu <string>\n");
12                      exit(1);
13              }
14              char *str = argv[1];
15              while (1) {
16                      Spin(1); // Repeatedly checks the time and
                                 returns once it has run for a second
17                      printf("%s\n", str);
18              }
19              return 0;
20      }
```

Simple Example(`cpu.c`): Code That Loops and Prints

Virtualizing the CPU (Cont.)

Execution result 2.

```
prompt> ./cpu A & ; ./cpu B & ; ./cpu C & ; ./cpu D &
[1] 7353
[2] 7354
[3] 7355
[4] 7356
A
B
D
C
A
B
D
C
A
C
B
D
...
```

NYU

Virtualizing the CPU (Cont.)

Execution result 2.

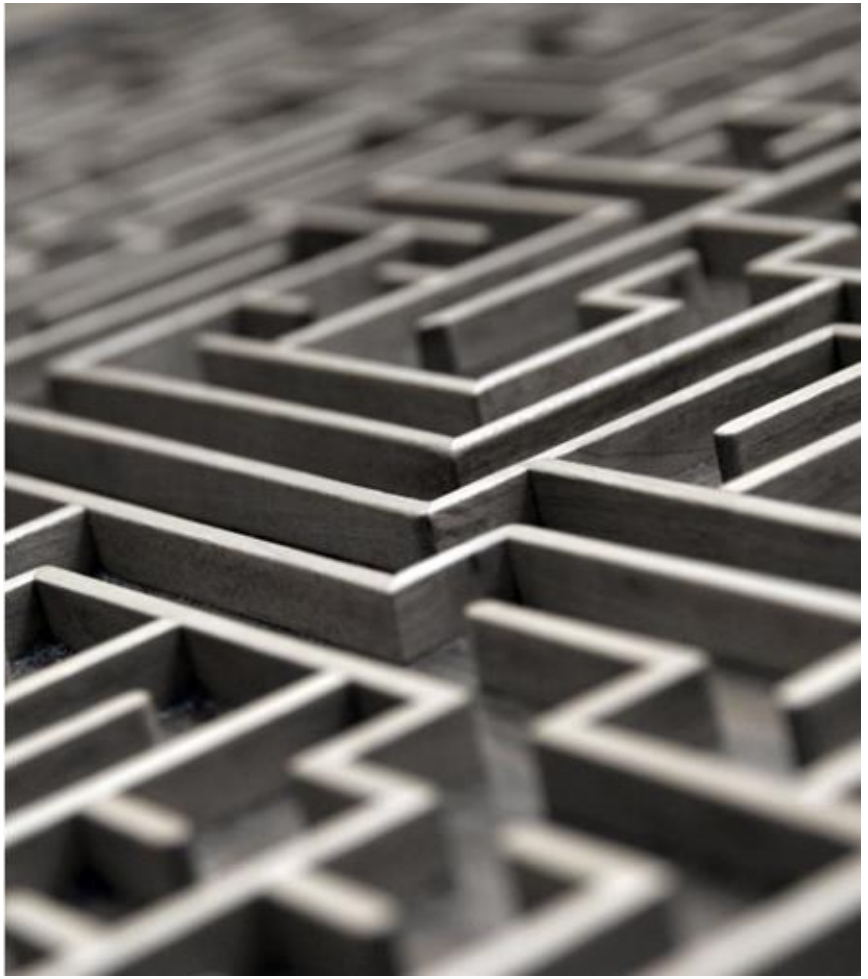
```
prompt> ./cpu A & ; ./cpu B & ; ./cpu C & ; ./cpu D &
[1] 7353
[2] 7354
[3] 7355
[4] 7356
A
B
D
C
A
B
D
C
A
C
B
D
...
```

Even though we have only **one processor**, all four programs seem to be running **at the same time**!

NYU

Challenge

- Provide the illusion of multiple CPUs
- More processes than processors
- This is called: Timesharing, scheduling, context switching



NYU

Goals for Solution

- **Transparent** to user processes
- **Pre-emptive** for user processes
- **Pre-emptive for Kernel** to keep system responsive

The Abstraction: Process

- A process is the basic unit of execution and an abstraction for a running program.
- Splitting up the execution of a system into processes allows:
  - Isolation between programs
  - The ability to simulate running multiple programs at once with only one CPU

Two different terminals start the same executable. Do we have one process or two?

One process, because the code is identical

0%

Two processes, because each run needs its own execution state

0%

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

Processes in Parallel

- It is confusing to reason about many events happening at once
- The process model lets us consider each process as a single, *sequential* execution
- (For now, just one CPU)

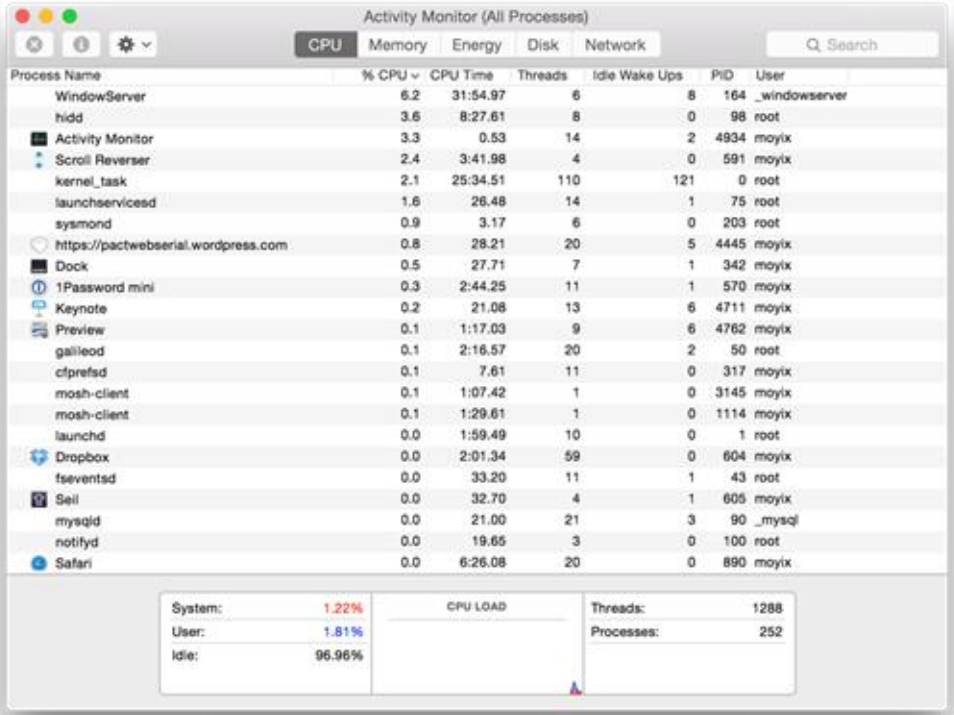
Processes != Programs

- A *program* = instructions + data
- A *process* is the OS abstraction for a running program, its resources, input/output, etc.
- Process = memory (instructions + data + stack)
- Think of a process as an instance of a program, just like an Object is an instance of a class.
- So: multiple copies of a program can be running in different processes

Process

- Each process has it’s own Machine State:
  - Memory
  - Registers (PC, SP, FP, etc)
  - Persistent storage (I/O)

Lots of Processes





Today

- Processes
- **Process States**
- Unix Processes
- XV6 Processes

Process API

- Create
- Destroy
- Wait
- Status
- Miscellaneous Control

Process Creation

- Processes can be created in many circumstances
  - At boot time
  - User request (e.g., typing a command)
  - A running program creates a new process on its own
- All of these are actually carried out by execution of a system call (with the exception of the first process, which is created directly by the kernel)

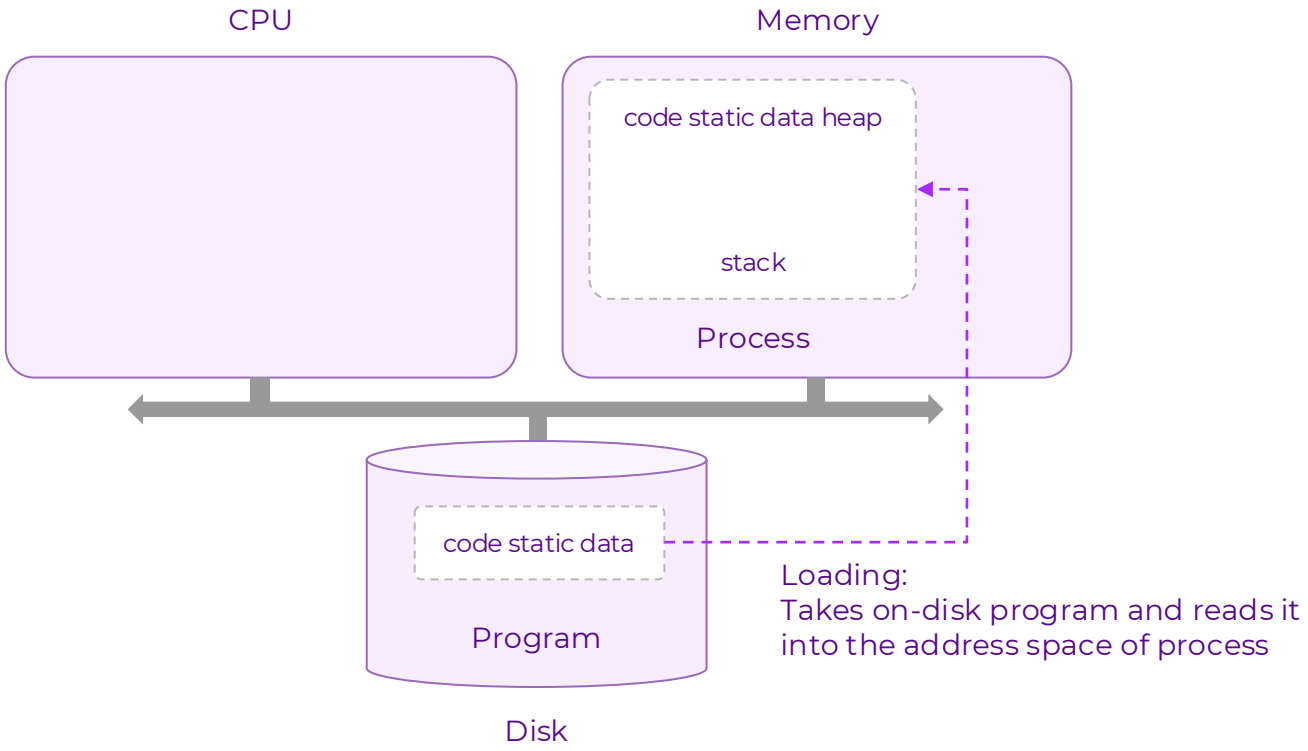


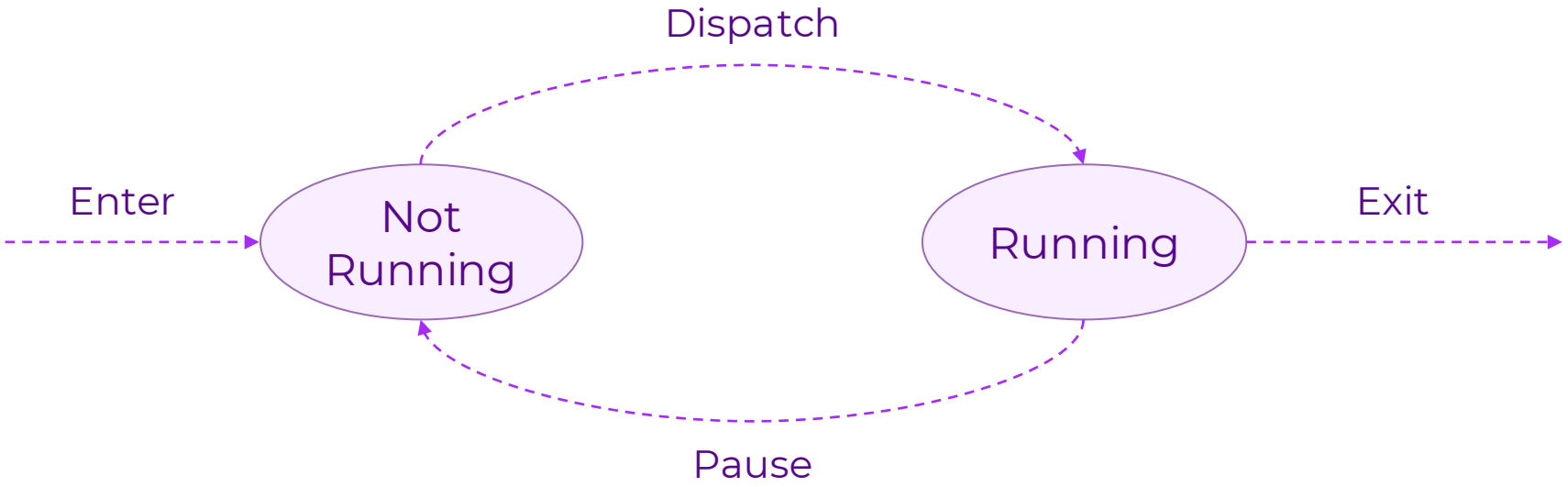
Figure 4.1: Loading: From Program To Process

Source: OS in 3 easy pieces.

Process Termination

- There must be a means for a process to indicate its completion
- For an interactive application, the action of the user will indicate when the process is completed (e.g. log off, quitting an application)
- A batch job should include a HALT instruction or an explicit OS service call for termination

Two-State Process Model



(a) State transition diagram

Untitled multiple choice question

A process is not running. Can the scheduler always run it next?

Vote yes or no.

Then produce one counterexample.

Be ready to say what state the two-state model is missing.

Yes

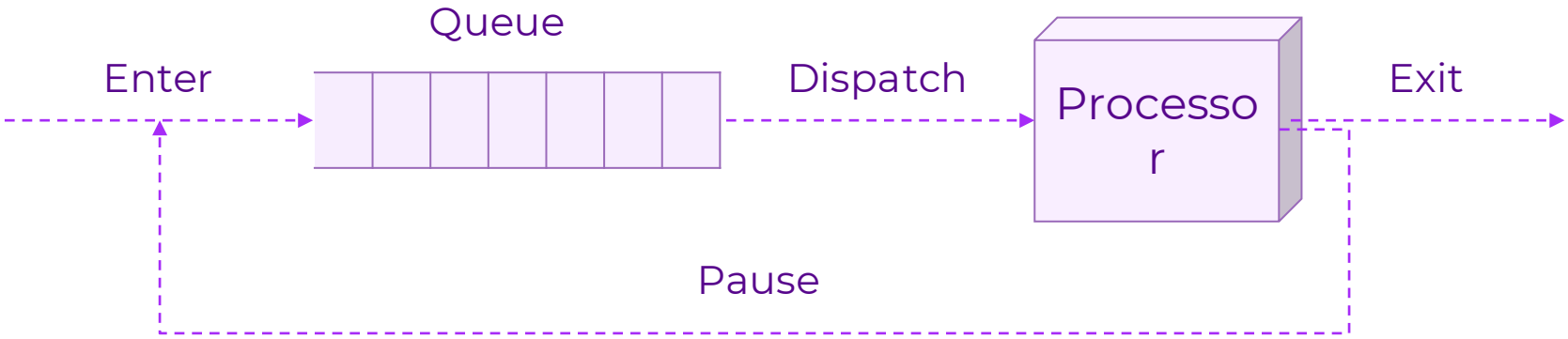
0%

No

0%

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

Two-State Process Model

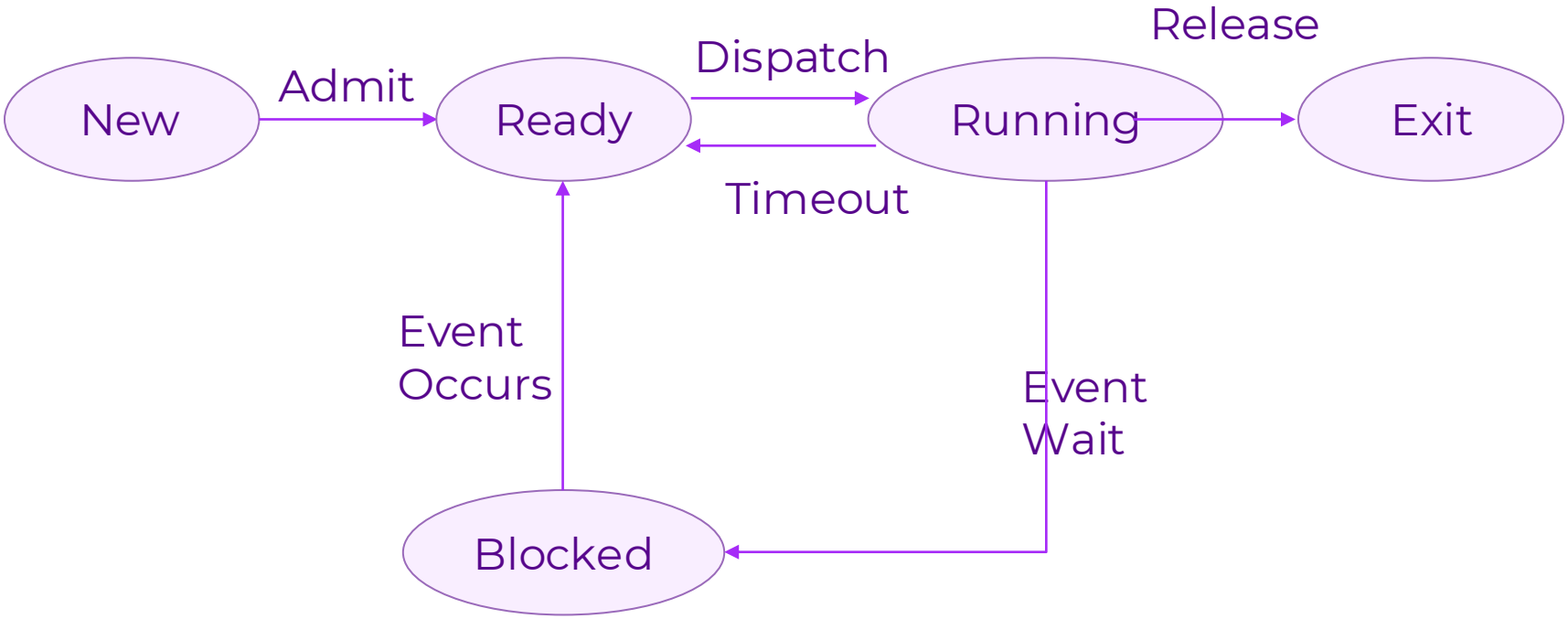


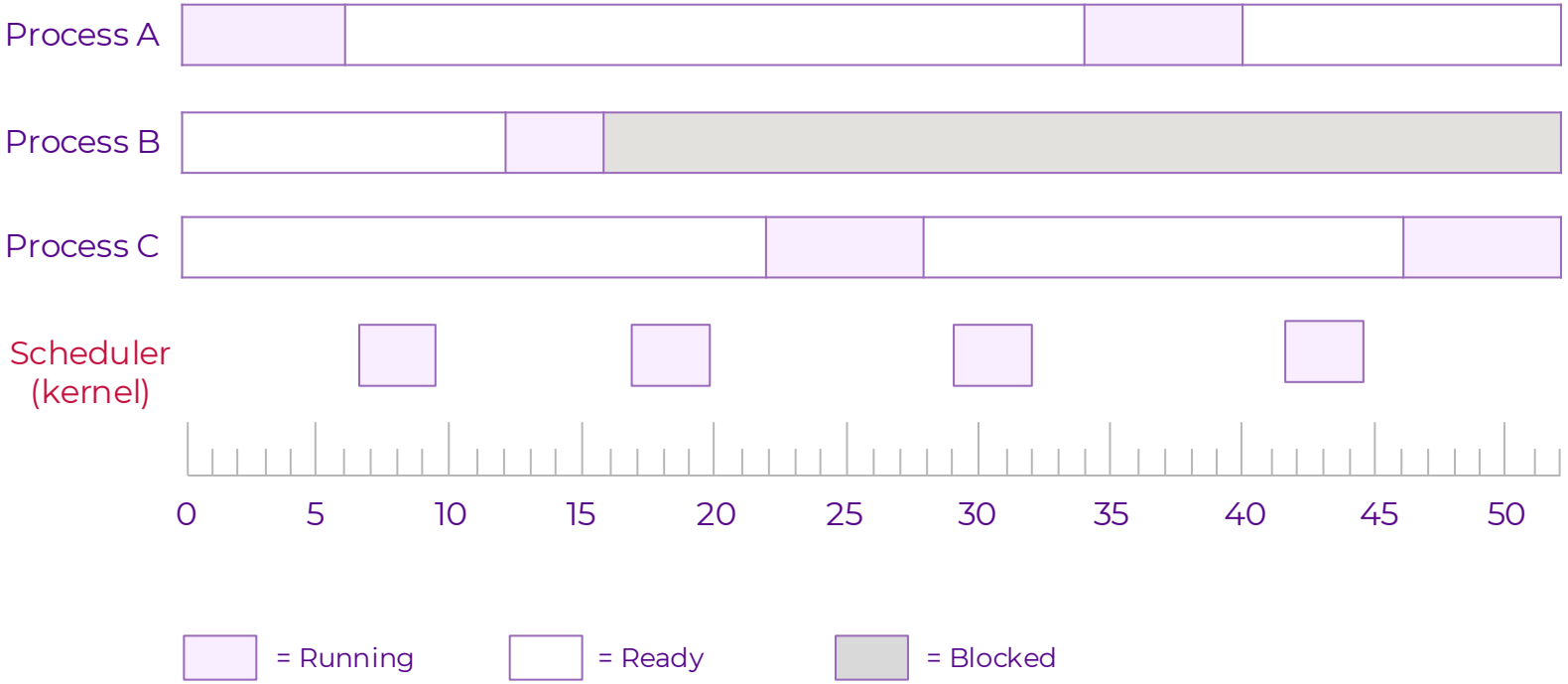
(a) Queuing diagram

Reasons for Proc Termination

- Normal Completion
- Bounds Violation
- Memory unavailable
- Others...

Five-State Process Model




Process A
Process B
Process C
Scheduler
(kernel)
0
5
10
15
20
25
30
35
40
45
50
= Running
= Ready
= Blocked

Mechanisms for Interrupting the execution of a process

- System Call: Way for user program to ask for OS service
- Exception: Illegal program action. Eg. Divide by zero, access memory for PTE not present, etc..
- Interrupt: Signal generated by hardware device. Eg. Clock chip, keyboard, etc.

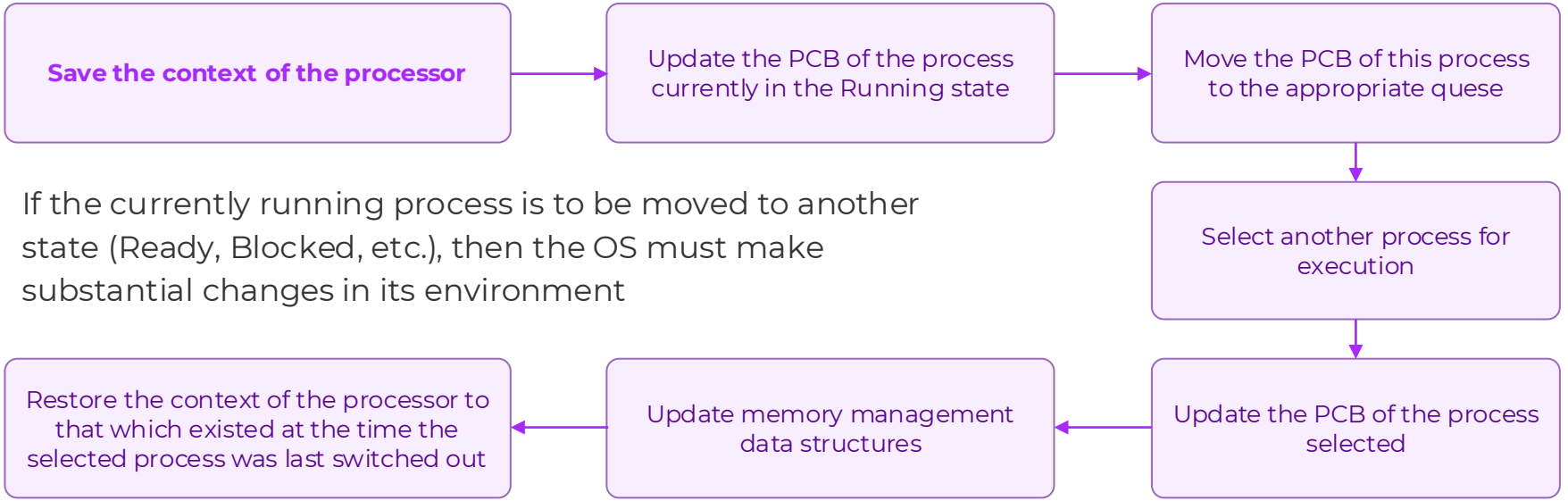
Role of the Process Control Block

The most important data structure in an OS

- Contains all of the information about a process that is needed by the OS
- Blocks are read and/or modified by virtually every module in the OS
- Defines the state of the OS

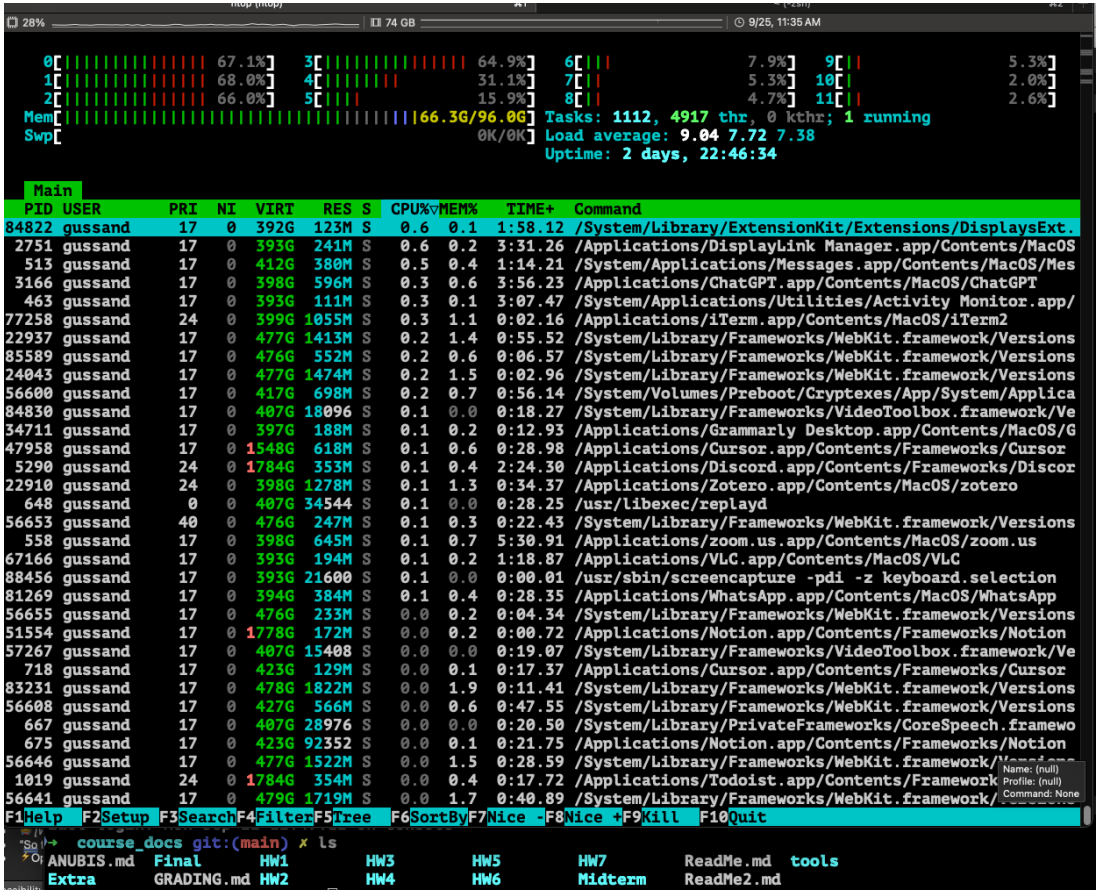
Change of Process State

The steps in a full process switch are:



If the currently running process is to be moved to another state (Ready, Blocked, etc.), then the OS must make substantial changes in its environment

Demo: htop


28%
74 GB
9/25, 11:35 AM
0[67.1%] 3[64.9%] 6[7.9%] 9[5.3%]
1[68.0%] 4[31.1%] 7[5.3%] 10[2.0%]
2[66.0%] 5[15.9%] 8[4.7%] 11[2.6%]
Mem[66.3G/96.0G]
Swp[0K/0K]
Tasks: 1112, 4917 thr, 0 kthr; 1 running
Load average: 9.04 7.72 7.38
Uptime: 2 days, 22:46:34
Main
PID USER PRI NI VIRT RES S CPU% MEM% TIME+ Command
84822 gussand 17 0 392G 123M S 0.6 0.1 1:58.12 /System/Library/ExtensionKit/Extensions/DisplaysExt.
2751 gussand 17 0 393G 241M S 0.6 0.2 3:31.26 /Applications/DisplayLink Manager.app/Contents/MacOS
513 gussand 17 0 412G 380M S 0.5 0.4 1:14.21 /System/Applications/Messages.app/Contents/MacOS/Mes
3166 gussand 17 0 398G 596M S 0.3 0.6 3:56.23 /Applications/ChatGPT.app/Contents/MacOS/ChatGPT
463 gussand 17 0 393G 111M S 0.3 0.1 3:07.47 /System/Applications/Utilities/Activity Monitor.app/
77258 gussand 24 0 399G 1055M S 0.3 1.1 0:02.16 /Applications/iTerm.app/Contents/MacOS/iTerm2
22937 gussand 17 0 477G 1413M S 0.2 1.4 0:55.52 /System/Library/Frameworks/WebKit.framework/Versions
85589 gussand 17 0 476G 552M S 0.2 0.6 0:06.57 /System/Library/Frameworks/WebKit.framework/Versions
24043 gussand 17 0 477G 1474M S 0.2 1.5 0:02.96 /System/Library/Frameworks/WebKit.framework/Versions
56600 gussand 17 0 417G 698M S 0.2 0.7 0:56.14 /System/Volumes/Preboot/Cryptexes/App/System/Applica
84830 gussand 17 0 407G 18096 S 0.1 0.0 0:18.27 /System/Library/Frameworks/VideoToolbox.framework/Ve
34711 gussand 17 0 397G 188M S 0.1 0.2 0:12.93 /Applications/Grammarly Desktop.app/Contents/MacOS/G
47958 gussand 17 0 1548G 618M S 0.1 0.6 0:28.98 /Applications/Cursor.app/Contents/Frameworks/Cursor
5290 gussand 24 0 1784G 353M S 0.1 0.4 2:24.30 /Applications/Discord.app/Contents/Frameworks/Discor
22910 gussand 24 0 398G 1278M S 0.1 1.3 0:34.37 /Applications/Zotero.app/Contents/MacOS/zotero
648 gussand 0 0 407G 34544 S 0.1 0.0 0:28.25 /usr/libexec/replayd
56653 gussand 40 0 476G 247M S 0.1 0.3 0:22.43 /System/Library/Frameworks/WebKit.framework/Versions
558 gussand 17 0 398G 645M S 0.1 0.7 5:30.91 /Applications/zoom.us.app/Contents/MacOS/zoom.us
67166 gussand 17 0 393G 194M S 0.1 0.2 1:18.87 /Applications/VLC.app/Contents/MacOS/VLC
88456 gussand 17 0 393G 21600 S 0.1 0.0 0:00.01 /usr/sbin/screencapture -pdi -z keyboard.selection
81269 gussand 17 0 394G 384M S 0.1 0.4 0:28.35 /Applications/WhatsApp.app/Contents/MacOS/WhatsApp
56655 gussand 17 0 476G 233M S 0.0 0.2 0:04.34 /System/Library/Frameworks/WebKit.framework/Versions
51554 gussand 17 0 1778G 172M S 0.0 0.2 0:00.72 /Applications/Notion.app/Contents/Frameworks/Notion
57267 gussand 17 0 407G 15408 S 0.0 0.0 0:19.07 /System/Library/Frameworks/VideoToolbox.framework/Ve
718 gussand 17 0 423G 129M S 0.0 0.1 0:17.37 /Applications/Cursor.app/Contents/Frameworks/Cursor
83231 gussand 17 0 478G 1822M S 0.0 1.9 0:11.41 /System/Library/Frameworks/WebKit.framework/Versions
56608 gussand 17 0 427G 566M S 0.0 0.6 0:47.55 /System/Library/Frameworks/WebKit.framework/Versions
667 gussand 17 0 407G 28976 S 0.0 0.0 0:20.50 /System/Library/PrivateFrameworks/CoreSpeech.framewo
675 gussand 17 0 423G 92352 S 0.0 0.1 0:21.75 /Applications/Notion.app/Contents/Frameworks/Notion
56646 gussand 17 0 477G 1522M S 0.0 1.5 0:28.59 /System/Library/Frameworks/WebKit.framework/
1019 gussand 24 0 1784G 354M S 0.0 0.4 0:17.72 /Applications/Todoist.app/Contents/Framework
56641 gussand 17 0 479G 1719M S 0.0 1.7 0:40.89 /System/Library/Frameworks/WebKit.framework/
F1Help F2Setup F3Search F4Filter F5Tree F6SortBy F7Nice - F8Nice + F9Kill F10Quit
Name: (null)
Profile: (null)
Command: None
course_docs git:(main) ✗ ls
ANUBIS.md Final HW1 HW3 HW5 HW7 ReadMe.md tools
Extra GRADING.md HW2 HW4 HW6 Midterm ReadMe2.md



Today

- Processes
- Process States
- **Unix Processes**
- XV6 Processes

Parent and Child Processes (UNIX)

- When one UNIX process creates another, the creator is the parent and the new process is its child
- This relationship creates a tree structure, rooted at the init process
- Processes are often managed as a group consisting of a single process and all its descendants
    - E.g., signals (such as SIGTERM, to request termination) are delivered to parent and all its descendants

Process Hierarchy (UNIX)

- In UNIX, each process (aside from init) has a parent process that created it
- This relationship creates a tree structure, rooted at the init process
- Processes are often managed as a group consisting of a single process and all its descendants
    - E.g., signals (such as SIGTERM, to request termination) are delivered to parent and all its descendants

Process Tree

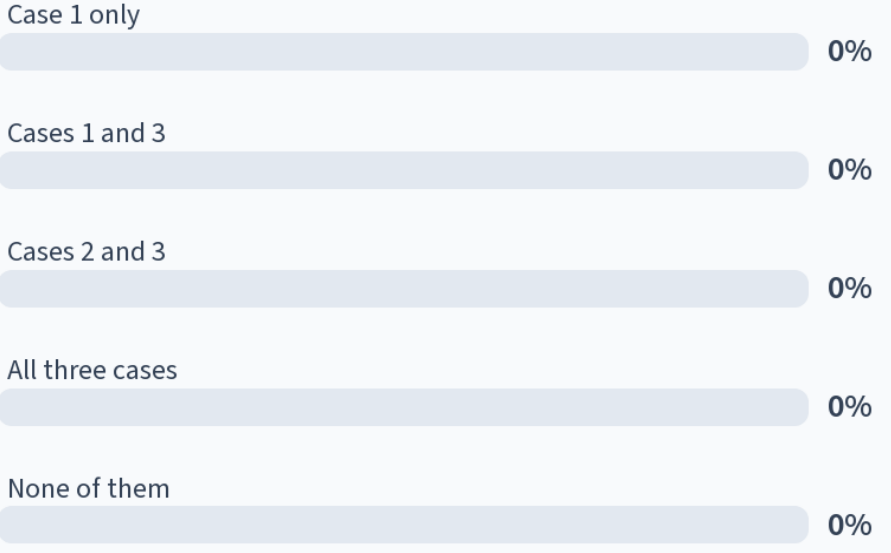
```
systemd─┬─ModemManager─┬─{gdbus}
        │              └─{gmain}
        ├─acpid
        ├─agetty
        ├─cron
        ├─dbus-daemon
        ├─dhclient
        ├─fail2ban-server───2*[{fail2ban-server}]
        ├─master─┬─pickup
        │        └─qmgr
        ├─mosh-server───bash───tmux
        ├─9*[mosh-server───bash]
        ├─ntpd
        ├─polkitd─┬─{gdbus}
        │         └─{gmain}
```

NYU

Case 1 Your shell stops a program you started Case 2 A normal user stops the system's login service Case 3 An administrator stops the same service

Should any process be allowed to terminate any other process?

- **Case 1** Your shell stops a program you started
- **Case 2** A normal user stops the system's login service
- **Case 3** An administrator stops the same service



Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

Killing Processes

- Processes can exit on their own using a system call
- One process can request that the OS terminate (kill) another process
    - Depending on the privilege of the requestor, this may or may not be granted

NYU

Killing Processes

- In most operating systems, process termination notifies the process to be terminated first so it can cleanly free resources
- More forceful methods exist:
  - kill -9
  - taskkill /f
- In all cases, the operating system will free the resources that it has granted to the process

The parent exits first. Must the child terminate too?

Yes, immediately

0%

No, it can continue

0%

Only if it is blocked

0%

None of the above

0%

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

Hierarchy and Termination (UNIX)

- If the parent process exits before its children, those children are said to be *orphaned*
- Orphaned child processes are *adopted* by init

CONTRADICTION

The child exits first. Why is it still visible?

- **The child has stopped executing.**
- The parent has not called wait().
- Which information must the kernel keep, and who still needs it?

NYU

Zombies (UNIX)

- *Zombies* are processes that have exited but still have an entry in the process table
- If a child process exits and the parent has not called wait() to get its exit status, the child becomes a zombie
- This is because the exit status is stored by the kernel-side data structure, and the kernel can't free it until it knows no one needs the exit status

```c
#include <stdlib.h>
#include <unistd.h>
#include <stdbool.h>
int main() {
    if (fork() == 0) {
        // Exit immediately in the child
        exit(1);
    }
    // Sleep forever without calling wait()
    // in the parent
    while(true);
    return 0;
}
```

zombie

USER	PID	%CPU	%MEM	VSZ	RSS	TT	STAT	STARTED	TIME	COMMAND
moyix	7622	100.0	0.0	2432748	548	s004	R+	8:05AM	0:15.23	./zombo
moyix	7623	0.0	0.0	0	0	s004	Z+	8:05AM	0:15.23	(zombo)

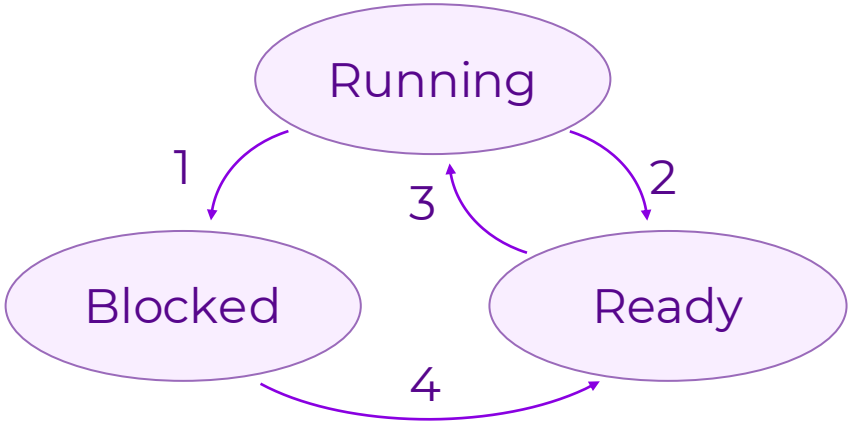
Process Hierarchy Anarchy (Windows)

- Windows does keep track of parent/child relationships
- These don't actually affect anything though
- Instead, when a process is created its parent gets a *handle* – a token that it can use to control the child
- Handles can be passed to other processes, and a process with sufficient privilege can obtain a handle to another process with OpenProcess

Process States

- Processes can generally be in one of three states:
  - *Running* – actively using the CPU right now
  - *Ready* – not currently running
  - *Blocked* – waiting for I/O
    - Even if the OS scheduler wants to run this process, it can't – nothing for it to do

Transitions



- Process blocks for input
- Scheduler picks another process
- Scheduler picks this process
- Input becomes available

NYU

I/O Waiting

- What causes processes to block?
  - Waiting for input
  - Explicit sleep
- The kinds of input can be things like network I/O, disk, waiting for the user to click on something, etc.

Process Anatomy: Kernel

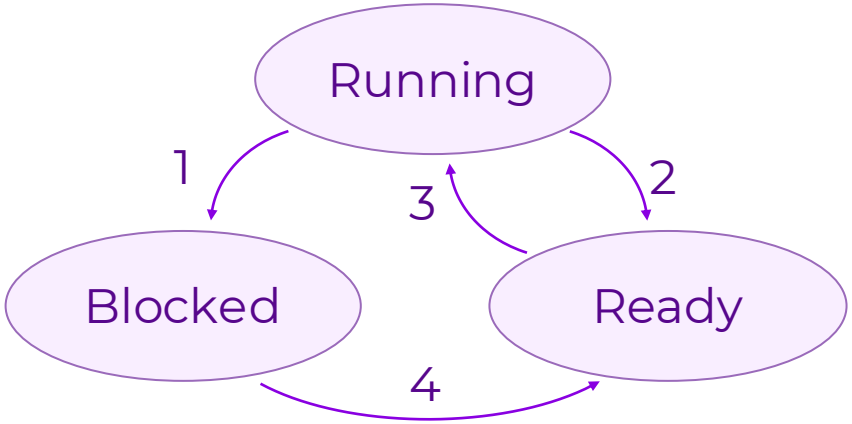
- The kernel tracks what processes are active in a *process table*
    - Note: in most OSes, this is not a literal array, but rather something like a doubly-linked list
- Each process gets a structure bit of metadata called the *process control block (PCB)*



Today

- Processes
- Process States
- Unix Processes
- **XV6 Processes**

Transitions



- Process blocks for input
- Scheduler picks another process
- Scheduler picks this process
- Input becomes available

NYU

Process Control Block in xv6

```
enum procstate { UNUSED, EMBRYO, SLEEPING, RUNNABLE, RUNNING, ZOMBIE };
// Per-process state
struct proc {
  uint sz;                     // Size of process memory (bytes)
  pde_t* pgdir;                // Page table
  char *kstack;                // Bottom of kernel stack for this process
  enum procstate state;        // Process state
  int pid;                     // Process ID
  struct proc *parent;         // Parent process
  struct trapframe *tf;        // Trap frame for current syscall
  struct context *context;     // swtch() here to run process
  void *chan;                  // If non-zero, sleeping on chan
  int killed;                  // If non-zero, have been killed
  struct file *ofile[NOFILE]; // Open files
  struct inode *cwd;           // Current directory
  char name[16];               // Process name (debugging)
};
```

NYU

xv6 Process Table

```
struct {
  struct spinlock lock;
  struct proc proc[NPROC];
} ptable;
```

Note: NPROC = 64

CONTRADICTION

Why does one thread need two stacks?

Predict what changes during a system call.

Which stack holds user frames?

Which stack holds kernel frames?

What would go wrong if user code could modify the kernel's stack?

Process Threads (aside)

- Each process has a thread of execution that executes the process instructions.
- Each thread has a PC and CPU State(Registers)
- Thread can be suspended and later resumed.
- Much of the state of a thread (local vars, ret address) stored on thread's stack.
- Each process has 2 stacks: user-stack and kernel stack (p->kstack)

Process Threads

- Process executing user instructions => user stack in use, kernel stack empty
- Process enters kernel thru sys_call or interrupt => kernel executes on the process's kernel stack. User stack contains saved data but not actively used.
- Kernel stack is separate and protected from user code.

Process Threads

- When a process makes a system call:
  - Processor switches to kernel stack
  - Raises hardware privilege level.
  - Starts executing the kernel instructions that implement the system call.

Process Threads

When System Call Completes:

- Kernel returns to user space
- hardware lowers privilege level.
- Switches back to user stack
- Resumes executing user instructions just after the system call

Kernel Stack

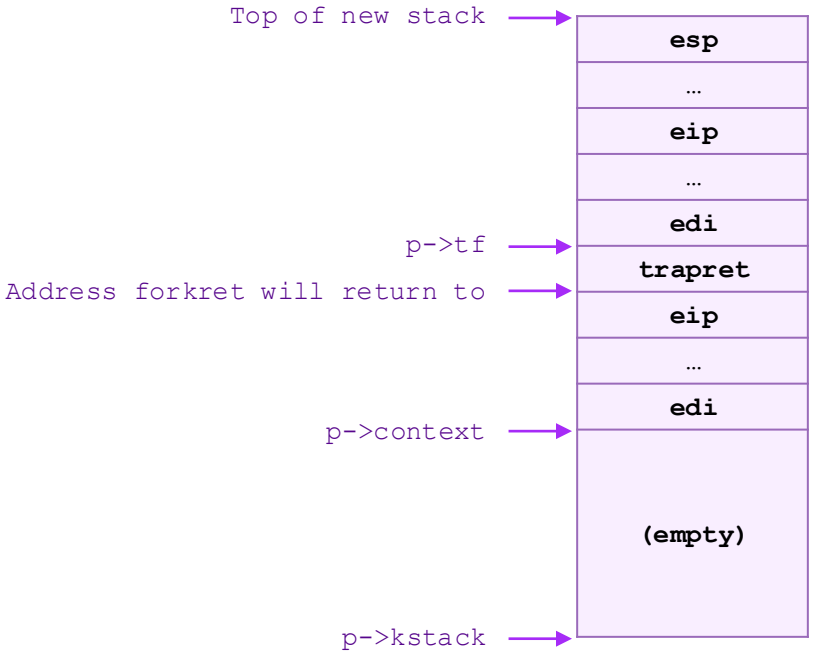
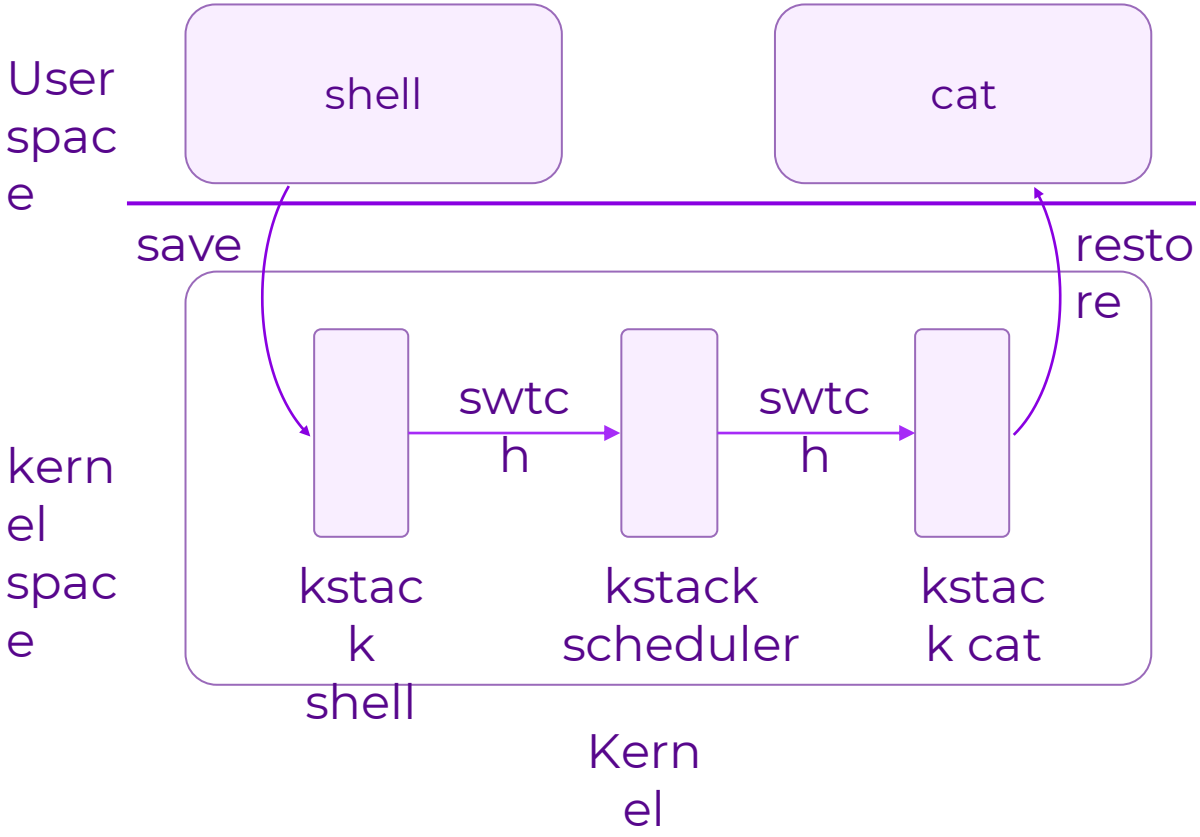


Figure 1-4. A new kernel stack.

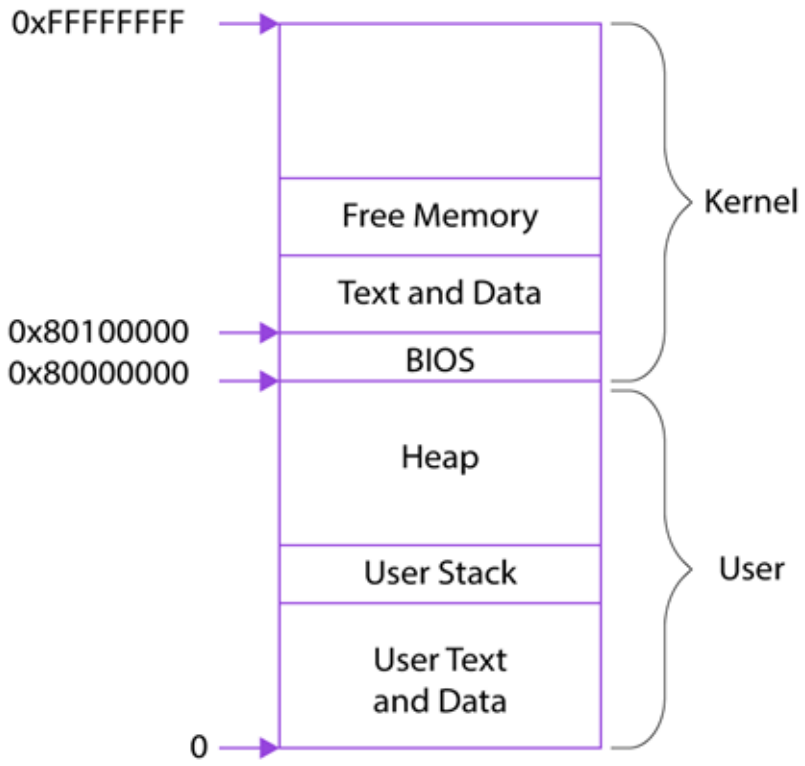

User space
shell
cat
save
restore
kernel space
swtch
swtch
kstack shell
kstack scheduler
kstack cat
Kernel

Create Proc

```
30  // Look in the process table for an UNUSED proc.
31  // If found, change state to EMBRYO and initialize
32  // state required to run in the kernel.
33  // Otherwise return 0.
34  static struct proc*
35  allocproc(void)
36  {
37    struct proc *p;
38    char *sp;
39
40    acquire(&ptable.lock);
41    for(p = ptable.proc; p < &ptable.proc[NPROC]; p++)
42      if(p->state == UNUSED)
43        goto found;
44    release(&ptable.lock);
45    return 0;
46
47  found:
48    p->state = EMBRYO;
49    p->pid = nextpid++;
50    release(&ptable.lock);
51
52    // Allocate kernel stack.
53    if((p->kstack = kalloc()) == 0){
54      p->state = UNUSED;
55      return 0;
56    }
57    sp = p->kstack + KSTACKSIZE;
```

```
52    // Allocate kernel stack.
53    if((p->kstack = kalloc()) == 0){
54      p->state = UNUSED;
55      return 0;
56    }
57    sp = p->kstack + KSTACKSIZE;
58
59    // Leave room for trap frame.
60    sp -= sizeof *p->tf;
61    p->tf = (struct trapframe*)sp;
62
63    // Set up new context to start executing at forkret,
64    // which returns to trapret.
65    sp -= 4;
66    *(uint*)sp = (uint)trapret;
67
68    sp -= sizeof *p->context;
69    p->context = (struct context*)sp;
70    memset(p->context, 0, sizeof *p->context);
71    p->context->eip = (uint)forkret;
72
73    return p;
74  }
```

xv6 Address Space Layout



NYU